Desktop Parking Slot Monitor — Project Report

Automated Software Testing — Prof. Lorenzo Bettini

Author: Wissal Barhoumi **University:** Università degli Studi di Firenze (UniFI) — Campus Morgagni **Repository:** https://github.com/wissalbarhoumi03-hub/parking-slot-monitor

Table of Contents

1. Introduction
2. Domain Model
3. Architecture
4. Technologies and Frameworks
5. Testing Strategy
6. The GUI and Multithreading
7. Build Automation and Continuous Integration
8. Problems Encountered and Solutions
9. Design Decisions and Deviations
10. How to Build and Run

1. Introduction

Desktop Parking Slot Monitor is a Java Swing desktop application that simulates and monitors the state of a small parking area. The user can add parking slots and mark them as occupied or free; each state change generates a *parking event* that is recorded with a timestamp. The application also displays the current parking status, shows the history of parking events, and computes the number of free and occupied slots.

The project was developed as a **low-rating** project for the *Automated Software Testing* exam. The application is written in **Java 17**, uses a **Swing** graphical user interface, and stores its data in a **MongoDB** database. The emphasis of the project is deliberately not on the complexity of the domain logic — kept intentionally simple — but on the rigorous application of the techniques taught in the course: Test-Driven Development, the testing pyramid (unit, integration, and end-to-end tests), code coverage, mutation testing, build automation with Maven, continuous integration with GitHub Actions, and code quality analysis with SonarCloud.

This report describes what was implemented, the applied techniques and frameworks, the main design and implementation choices, the problems encountered during development and how they were solved, and the development of the most interesting parts of the project.

2. Domain Model

The domain of the application is intentionally small and revolves around two entities: `ParkingSlot` and `ParkingEvent`.

ParkingSlot

`ParkingSlot` is the main entity of the application. It is identified by a `String id` and holds a single `boolean` field, `occupied`, that indicates whether the slot is currently taken. Representing the state as a plain boolean was a deliberate choice to keep the domain logic minimal, in line with the low-rating nature of the project.

The class is a plain Java object with a no-argument constructor (required so that the MongoDB driver can instantiate it during deserialization) and a full constructor. It exposes standard getters and setters, and overrides `equals`, `hashCode`, and `toString`. These methods are generated with `java.util.Objects` rather than with any external library such as Lombok, in accordance with the exam constraints. The `equals` and `hashCode` methods are essential because the tests and the Swing list rely on value equality to compare slots, and `toString` defines how a slot is rendered in the user interface.

ParkingEvent

`ParkingEvent` is a second entity, introduced to support the history feature declared in the project proposal. It records a single change of state of a slot and holds three fields: the `slotId` of the affected slot, the `boolean occupied` value resulting from the change, and a `String timestamp`. Every time a slot is marked as occupied or free, a corresponding `ParkingEvent` is created and stored, so that the application can later display the full history of what happened in the parking area.

The timestamp is stored as a `String` rather than as a dedicated date/time type. This keeps persistence straightforward — the value is produced by the controller and stored and retrieved verbatim — and avoids introducing any additional mapping logic, again consistent with the goal of keeping the project simple. Like `ParkingSlot`, `ParkingEvent` is a plain Java object with a no-argument constructor, a full

constructor, standard accessors, and `equals`, `hashCode`, and `toString` implemented through `java.util.Objects`.

The two entities are only loosely coupled: a `ParkingEvent` references a slot through its `slotId` value, but there is no object reference between them and no foreign-key constraint. This keeps each entity independent and easy to persist in its own MongoDB collection.

---

3. Architecture

The application follows a layered, Model-View-Controller architecture, organized into clearly separated packages:

- `com.example.parking.model` — the domain entities (`ParkingSlot`, `ParkingEvent`);
- `com.example.parking.repository` — the repository interfaces;
- `com.example.parking.repository.mongo` — the MongoDB implementations of those interfaces;
- `com.example.parking.controller` — the application logic;
- `com.example.parking.view` — the view interface;
- `com.example.parking.view.swing` — the Swing implementation of the view;
- `com.example.parking.app.swing` — the application entry point.

The repository layer

Persistence is hidden behind two interfaces. `ParkingSlotRepository` declares the operations needed on slots: `findAll`, `findById`, `save`, and `delete`. `ParkingEventRepository` is smaller and declares only `findAll` and `save`, because events are only ever created and listed, never updated or deleted. Depending on an interface rather than on a concrete class means the controller is completely decoupled from MongoDB: in unit tests the repositories are replaced by mocks, and the concrete MongoDB implementations are exercised separately by integration tests.

The controller layer

`ParkingController` contains the application logic and is the only component that both talks to the repositories and drives the view. It receives a `ParkingView`, a `ParkingSlotRepository`, and a `ParkingEventRepository` through its constructor, which makes its dependencies explicit and easy to substitute in tests. This constructor-based wiring is the manual dependency injection; no dependency-injection framework was introduced, in keeping with the low-rating scope of the project.

The view layer

The `ParkingView` interface defines how the controller communicates with the user interface, independently of any GUI toolkit. It declares `showAllSlots` and `showHistory` to display the current slots and the event history, `showCounts` to display the number of free and occupied slots, `slotAdded` and `slotUpdated` to reflect the creation and the state change of a slot, and `showError` to report problems. The `slotUpdated` notification reflects a change in the state of a slot rather than its removal, since in this application slots are updated rather than deleted, while `showHistory` and `showCounts` support the history and counting features.

The concrete implementation, `ParkingSwingView`, is a Swing window that implements this interface. Because it depends only on the `ParkingView` interface, the controller is unaware that the user interface is built with Swing, and the same controller could in principle drive a different implementation of the view.

Flow of control

A typical interaction flows in one direction and back: the user acts on the Swing view, the view delegates to the controller, the controller reads or writes through the repositories, and then the controller calls back into the view (through the `ParkingView` interface) to reflect the result. This one-directional collaboration between view, controller, and repository keeps each layer focused on a single responsibility.

---

4. Technologies and Frameworks

All the technologies used in this project are the ones presented in the course; no additional framework was introduced. This section describes how each of them is used in the parking application.

Language and user interface

The application targets **Java 17** and its user interface is built with **Swing**, part of the standard Java library. Swing was chosen because it allows the graphical interface to be tested at the unit, integration, and end-to-end levels.

Persistence

Data is stored in **MongoDB**, accessed through the **MongoDB Java driver**. Each entity has its own collection, and the repository implementations in `repository.mongo` translate between the Java objects and the MongoDB documents. MongoDB was selected because its document model fits the two simple entities of the domain without requiring any object-relational mapping.

The application entry point

The `ParkingSwingApp` class is the entry point and uses **Picocli** to parse command-line options. This makes the MongoDB connection parameters — host, port, database name, and collection names — configurable from the command line, with sensible defaults for local execution, turning the application into a configurable command.

Testing libraries

The tests rely on several libraries, each used for a specific purpose:

- **JUnit 4** is the testing framework used for all unit, integration, and end-to-end tests.
- **Mockito** is used in the controller unit tests to replace the repositories and the view with mocks, so that the controller logic can be verified in isolation, including the order of the interactions.
- **AssertJ Swing** is used to drive and inspect the Swing interface from the tests: it locates components, simulates user actions such as clicks and text input, and asserts on the state of the widgets.
- **Testcontainers** starts a real MongoDB instance inside a Docker container for the integration and end-to-end tests, so that the MongoDB repository implementations and the whole application are tested against a genuine database rather than a simulated one.
- **mongo-java-server** provides an in-memory MongoDB replacement used in some of the view integration tests, where a full container is not necessary and a lighter in-memory backend is enough.
- **Awaitility** is used to wait, in a controlled way, for the results of the background threads started by the GUI, so that assertions are only checked once the asynchronous work has completed.

Build, coverage, mutation, and quality tools

The build is managed with **Maven**. **JaCoCo** measures code coverage, **PIT** performs mutation testing on the business-logic classes, **Coveralls** publishes the coverage results, and **SonarCloud** analyzes the overall code quality. These tools are described in the build and continuous-integration section of this report.

---

5. Testing Strategy

Testing is the core of this project, and the whole application was built by following Test-Driven Development: production code was written only to make a previously written, failing test pass. The tests are organized according to the testing pyramid, with a broad base of fast unit tests, a smaller layer of integration tests, and a few end-to-end tests at the top. To keep the three levels clearly separated, the project uses three distinct source folders — `src/test/java` for unit tests, `src/it/java` for integration tests, and `src/e2e/java` for end-to-end tests — configured through the Maven build.

Unit tests

Unit tests form the base of the pyramid and run in complete isolation, without any database or real user interface. The controller is tested by replacing its collaborators — the two repositories and the view — with Mockito mocks; this makes it possible to verify not only the results but also that the controller interacts with its collaborators in the expected way and in the expected order. The MongoDB repositories are unit-tested against an in-memory MongoDB backend, and the Swing view is unit-tested with AssertJ Swing, which allows the widgets and their behavior to be checked without starting the full application.

Integration tests

Integration tests verify that the components work correctly together with a real database. The MongoDB repository implementations are tested against a genuine MongoDB instance started in a Docker container through Testcontainers, ensuring that the mapping between Java objects and MongoDB documents behaves as expected. At the view level, integration tests wire the real Swing view together with the real controller and a database, and check that a user action propagates correctly all the way through the layers.

End-to-end tests

End-to-end tests sit at the top of the pyramid and exercise the whole application exactly as a user would. The application is launched as a whole — connected to a MongoDB instance started with Testcontainers — and AssertJ Swing drives the real graphical interface, clicking buttons and inspecting the resulting state. These tests are the slowest and the fewest, but they give confidence that all the pieces work together correctly, from the GUI down to the database.

Code coverage

Code coverage is measured with JaCoCo and reaches 100% of instructions and branches. Coverage was never treated as a goal in itself but as a tool to reveal untested code; the two entity classes and the application entry point, whose behavior is trivial or exercised only through the end-to-end tests, are excluded from the coverage computation. The coverage results are also published to Coveralls and shown as a badge in the repository.

Mutation testing

Mutation testing is performed with PIT on the classes that contain the actual business logic — the controller and the MongoDB repositories — using the corresponding unit tests. PIT systematically introduces small changes (mutations) into this code and checks that at least one test fails for each of them; the suite kills 100% of the generated mutants, which confirms that the tests genuinely verify the behavior of the code rather than merely executing it. The integration, end-to-end, and GUI tests are excluded from the mutation run, so that mutation testing focuses on the business logic and does not depend on Docker or on the graphical environment.

---

6. The GUI and Multithreading

The graphical user interface is the part of the application where the most interesting technical problems arise, because a Swing interface is inherently multithreaded and must remain responsive while the application talks to the database.

The Swing view

`ParkingSwingView` is a `JFrame` that implements the `ParkingView` interface. It contains a text field for entering a slot identifier, buttons to add a slot and to mark it as occupied or free, two lists — one showing the current slots and one showing the history of events — a label showing the free and occupied counts, an error label, and two buttons that display the history and the counts on demand. The components are arranged with a grid-based layout, and each interactive component is given an explicit name so that it can be located reliably by the AssertJ Swing tests.

Keeping the interface responsive

Swing is single-threaded: all updates to the interface must happen on the Event Dispatch Thread (EDT). At the same time, the operations triggered by the buttons involve the database and may take time; running them on the EDT would freeze the interface. The application therefore follows a clear threading discipline: when a button is pressed, the corresponding call to the controller is executed on a separate background thread, so that the potentially slow database operation does not block the interface. Conversely, every method of the view that updates the widgets wraps its work in `SwingUtilities.invokeLater`, so that the update is always performed on the EDT, even when it is requested from a background thread. This two-way discipline — controller calls off the EDT, view updates back on the EDT — keeps the interface both responsive and thread-safe.

Race conditions

Because several background threads may run concurrently, the application is exposed to race conditions. The scenario considered is the concurrent addition of the same slot: two threads may both check that a slot does not yet exist and then both create it, resulting in a duplicate. This situation is reproduced deliberately in dedicated tests that launch several threads at once and then wait, using Awaitility, until all of them have finished.

Two complementary measures address the problem. First, at the application level, the methods of the controller that modify the parking state are declared `synchronized`, so that on a single controller instance these operations cannot interleave. Second, at the database level — where `synchronized` is not enough, because different controller instances or different runs of the application could still collide — a unique index is created on the slot identifier, so that MongoDB itself rejects any attempt to insert a duplicate slot. This unique-index safeguard is a standard uniqueness constraint on a field, not a multi-document transaction, which keeps the solution simple and within the scope of the course.

The history and counts features

The buttons that show the history and the counts follow the same delegation pattern as the other buttons: pressing them runs the corresponding controller call on a background thread, and the controller then calls back into the view to display the result. The history display clears its list before refilling it, so that pressing the button several times does not accumulate duplicate entries. These two features were verified at all three levels of the testing pyramid, like the rest of the application.

---

7. Build Automation and Continuous Integration

The project is fully automated with Maven, and every change is validated by a continuous integration pipeline before being merged.

The Maven build

A single Maven command compiles the code and runs the complete test suite — unit, integration, and end-to-end tests. The three kinds of tests live in three separate source folders, which are added to the build with the `build-helper` plugin. Unit tests run during the standard test phase, while integration and end-to-end tests are executed by the `failsafe` plugin during the integration-test phase, so that the slower tests requiring Docker are clearly separated from the fast unit tests.

Several Maven profiles keep the build flexible. A `jacoco` profile enables the code-coverage report; a `mutation-testing` profile runs PIT; and a `skipTestcontainers` profile allows the tests that need Docker to be skipped, which is useful on environments where Docker is not available. Coverage exclusions, the list of test folders analyzed by SonarCloud, and the rules to ignore during the SonarCloud analysis are all configured as properties in the POM.

Continuous integration on GitHub Actions

Continuous integration is provided by GitHub Actions, with two workflows. The main workflow runs on Linux and is triggered on every push and on every pull request. It uses a build matrix to run the whole build on two Java versions, 17 and 21, which verifies that the project builds and its tests pass on more than one Java runtime. Because the GUI tests need a display, the Linux workflow runs them under a virtual framebuffer (Xvfb). The coverage submission to Coveralls and the SonarCloud analysis are performed only once, on the Java 17 job, to avoid submitting the same results twice from the two matrix jobs.

A second workflow runs the build on Windows and macOS, but only on pull requests. On these two operating systems the tests that rely on Docker are skipped through the `skipTestcontainers` profile, following the recommendation in the guidelines to focus the Docker-based tests on Linux while still checking that the rest of the build works on the other operating systems.

Quality gates in the pipeline

Each pull request must pass all the checks before being merged: the build and the full test suite on both Java versions on Linux, the build on Windows and macOS, the coverage submission to Coveralls, and the SonarCloud analysis. The repository was developed with one pull request per feature, each validated by this pipeline, which is reflected in the Git history and in the three badges shown in the README: the build status (linking to the GitHub Actions logs), the code coverage from Coveralls, and the quality gate from SonarCloud.

Code quality with SonarCloud

SonarCloud analyzes the code on every run of the Linux workflow and enforces a quality gate. The project reaches a passing quality gate with no open issues, 100% coverage as reported to SonarCloud, and no duplicated code. Reaching this result required both correcting real issues in the code and, for a small number of false positives, disabling specific rules with an explicit justification — for example, the naming rules triggered by the GUI code generated with the window builder, and the "tests should contain assertions" rule triggered by the AssertJ Swing tests, whose assertions SonarCloud does not recognize. Each exclusion is documented in the POM with its motivation, and none of them was used merely to reach zero technical debt.

---

8. Problems Encountered and Solutions

Several concrete problems were encountered during the development of the project. This section describes the most significant ones and how they were solved, since they illustrate the kind of difficulties that arise when combining a graphical interface, a database, concurrency, and a continuous-integration pipeline.

Flaky GUI tests caused by the keyboard layout

Some GUI tests that type text into the interface occasionally failed on the local machine, with the injected character not matching the expected one — for instance, a digit being replaced by a symbol. The cause was not in the code but in the keyboard layout of the development machine, which the AssertJ Swing robot did not map as expected. Changing the keyboard layout removed the problem locally, and on the continuous-integration server the issue does not occur at all, because the Linux workflow runs the GUI tests under a virtual framebuffer with a predictable keyboard mapping.

Duplicate submissions to Coveralls from the build matrix

After introducing the Java build matrix, the pipeline started to fail intermittently on the coverage step, with Coveralls reporting that the build was already closed. The reason was that both matrix jobs — Java 17 and Java 21 — were submitting their coverage to Coveralls for the same build, and the second submission arrived after the build had been closed by the first. The solution was to submit the coverage only from the Java 17 job, in the same way the SonarCloud analysis is run only once; the other matrix job now builds and tests the project without submitting coverage.

Mutation testing running the Docker-based tests

When mutation testing was run after the integration and end-to-end source folders had been added to the build, PIT failed before mutating anything, reporting that the test suite was not green. The investigation showed that PIT was trying to execute all the tests, including the integration and end-to-end tests that require Docker, which failed in the PIT execution environment. The solution was to restrict mutation testing to the business logic and its unit tests by excluding the integration, end-to-end, and GUI test classes from the PIT run. This keeps mutation testing focused on the classes that contain the actual logic, and it restored a green suite and a 100% mutation score.

History and counts features not reachable from the interface

While reviewing the application, it became clear that the methods computing the event history and the free/occupied counts existed and were unit-tested, but were never triggered from the running application: no button invoked them, so two of the features declared in the proposal were not actually usable. Two dedicated buttons were added to the interface, following the same delegation pattern as the existing buttons and developed test-first, so that the history and the counts became reachable by the user. The new buttons were then covered at all three levels of the testing pyramid.

Event accumulation across integration tests

When the integration tests for the history feature were added, one of them failed because the history contained more events than expected. The cause was that the test setup emptied only the slot collection between tests, leaving the events created by previous tests in the database. The setup was changed to drop the whole database before each test, following the same clean-database approach used elsewhere in the project, which made every integration test start from a known, empty state.

Displaying the history in the interface

A smaller usability problem appeared when testing the application manually: the history was displayed correctly but in an area too small to be readable. This was solved by giving the history list a proper vertical weight in the layout and enlarging the window, so that the history is clearly visible. This change concerned only the layout and did not affect any test.

---

9. Design Decisions and Deviations

This section documents the main design and implementation decisions taken during the project, together with the reasons behind them, so that each choice can be justified.

Domain model

The state of a parking slot is represented as a single `boolean` field, which keeps the domain deliberately simple. A second entity, `ParkingEvent`, records each change of a slot's state with a timestamp, supporting the history feature declared in the approved project proposal. The timestamp is stored as a plain `String`, produced by the controller, to avoid introducing any date/time mapping logic.

Operations

The application offers three write operations — "add", "mark occupied", and "mark free". Marking a slot updates its state and records an event rather than removing the slot, which is why the view is notified through `slotUpdated` rather than a removal notification. Two additional read-only operations display the event history and compute the number of free and occupied slots. All of them are built from the same simple building blocks — a repository query, a list, and a label — without introducing any new technology.

Concurrency and the unique index

The multithreading behavior was implemented in full rather than simplified away. In addition to declaring the state-changing controller methods `synchronized`, a unique index on the slot identifier was created in MongoDB to prevent duplicate slots under concurrent insertions. It is important to note that this is a standard uniqueness constraint on a single field, not a multi-document transaction; this distinction was made deliberately to keep the project within its intended low-rating scope.

Testing decisions

For the integration tests of the repositories and for the end-to-end tests, a real MongoDB instance is started with Testcontainers, in preference to connecting to a manually started local database. This is more in line with the exam requirement that Docker be used through Testcontainers, and it makes the tests self-contained. A lighter in-memory MongoDB backend is used in some of the view integration tests, where a full container is not needed. For end-to-end tests, the two slot lists in the interface are looked up explicitly by name, since the window contains more than one list.

Build and continuous integration

Both the Coveralls submission and the SonarCloud analysis are run only on the Java 17 job of the build matrix, to avoid submitting the same results twice. The build matrix itself runs on Java 17 and Java 21 to check runtime compatibility, while the compilation target remains Java 17. A small number of SonarCloud rules are disabled with explicit justifications, limited to genuine false positives such as the naming of GUI code generated by the window builder and the assertions of AssertJ Swing tests that SonarCloud does not recognize. Mutation testing is deliberately narrowed to the business-logic classes and their unit tests.

Scope decisions

A few techniques available in the course were intentionally not included, in order to keep the project within its low-rating scope. The application itself is not packaged into a Docker image: Docker is required by the guidelines for the tests, which is fully satisfied through Testcontainers, whereas containerizing a Swing application would require a graphical environment and goes beyond what is needed. Behavior-driven testing with Cucumber and dependency injection with a framework such as Google Guice were also left out on purpose, as both would move the project toward a higher-rating scope; dependency injection is instead performed manually through constructors.

---

10. How to Build and Run

The project is designed to be built and run with a minimal number of commands. The instructions below assume that Java 17 and Docker

are installed on the machine, as stated in the exam guidelines.

Prerequisites

- Java 17
- Docker running (required for the integration and end-to-end tests, which start MongoDB automatically through Testcontainers)

Building and testing

The whole project — compilation, unit tests, integration tests, and end-to-end tests — is built and verified from the repository root with a single command:

```
mvn -f parking-slot-monitor/pom.xml clean verify
```

To also generate the JaCoCo code-coverage report, the `jacoco` profile is enabled:

```
mvn -f parking-slot-monitor/pom.xml clean verify -Pjacoco
```

The coverage report is then available under `parking-slot-monitor/target/site/jacoco/index.html`.

On a machine where Docker is not available, the tests that require it can be skipped with the `skipTestcontainers` profile, so that the rest of the build still runs:

```
mvn -f parking-slot-monitor/pom.xml clean verify -PskipTestcontainers
```

Running the mutation testing

Mutation testing is run on the business-logic classes with the `mutation-testing` profile:

```
mvn -f parking-slot-monitor/pom.xml test org.pitest:pitest-maven:mutationCoverage -Pmutation-testing
```

Running the application

The application needs a running MongoDB instance. A local one can be started with Docker:

```
docker run -d -p 27017:27017 --name mongo mongo:5
```

The application is then started from the `parking-slot-monitor` directory:

```
mvn exec:java "-Dexec.mainClass=com.example.parking.app.swing.ParkingSwingApp"
```

By default the application connects to MongoDB on `localhost:27017`. The connection can be customized through the command-line options `--mongo-host`, `--mongo-port`, `--db-name`, `--slot-collection`, and `--event-collection`.